

Ferramenta PUPPET

Documentação feita pelo grupo 2

Recife

2026

Sumário

1. Introdução
2. Divisão de Tarefas do Grupo
3. Principais Usos para o Mercado
4. Principais Comandos
5. Teste Manifest Puppet
6. Referências

1.Introdução

Puppet é uma ferramenta de automação de infraestrutura e gestão de configuração (Configuration Management), utilizada para definir, aplicar e manter o estado desejado de servidores e demais recursos de TI de forma automatizada e consistente. Foi lançada em 2005 e atualmente é desenvolvida e mantida pela Perforce Software, que adquiriu a Puppet em 2022.

O princípio central do Puppet é a abordagem declarativa: em vez de o administrador escrever um roteiro de passos ("como fazer"), ele descreve o estado final desejado do sistema ("o que deve existir"), e o Puppet se encarrega de calcular e aplicar as mudanças necessárias para alcançar e manter esse estado — corrigindo automaticamente qualquer desvio de configuração (configuration drift) que venha a ocorrer.

1.1 Características Principais

- Infraestrutura como Código (IaC): configurações são escritas em arquivos de texto chamados manifests (extensão .pp), versionáveis em sistemas como Git.
- Modelo declarativo: o foco está no resultado final, não na sequência de comandos.
- Idempotência: aplicar o mesmo manifesto múltiplas vezes produz sempre o mesmo resultado, sem efeitos colaterais indesejados.
- Arquitetura agente-servidor (Master-Agent): nós agentes se comunicam periodicamente com um Puppet Server central para obter o "catálogo" de configurações a aplicar; também é possível o uso sem servidor (modo standalone, via puppet apply).
- Linguagem própria (Puppet DSL): baseada em Ruby, usada para descrever recursos como pacotes, serviços, arquivos e usuários.
- Módulos reutilizáveis: disponíveis no repositório público Puppet Forge, contendo configurações prontas para tecnologias como Apache, MySQL, Nginx, entre outras.

1.2 Componentes da Arquitetura

- Puppet Server (Master): compila os manifests em catálogos e os distribui aos agentes.
- Puppet Agent: instalado nos nós gerenciados; aplica periodicamente o catálogo recebido.
- Facter: ferramenta que coleta informações (facts) sobre o sistema — SO, IP, memória, etc. — usadas nos manifests.
- Hiera: sistema de busca de dados hierárquico, usado para separar dados de configuração da lógica do código.
- PuppetDB: banco de dados que armazena os dados de catálogos, relatórios de execução e facts, permitindo consultas e auditoria.

2. Divisão de Tarefas do Grupo

Para a elaboração desta documentação, o trabalho foi organizado nas seguintes frentes, permitindo que cada integrante do grupo aprofunde-se em uma etapa específica do processo, com posterior integração e revisão conjunta do material final.

Divisão de tarefas	Alunos	Descrição da Atividade
Gerente de Repositório	Otavio Souza Leao de Barros	Criar o repositório inicial no GitHub, configurar a estrutura base e auxiliar o resto do grupo na criação de branches e commits para evitar conflitos. Vai garantir que a entrega final no GitHub esteja impecável.
Redator do Guia Prático (README.md) - Primeira parte	Leonardo Antonio da Silva	Leonardo será responsável pela documentação prática no GitHub. Vai escrever em Markdown o passo a passo de como fazer a instalação do Puppet e a configuração inicial. O foco aqui é prático, sem teoria pesada..
Redator do Guia Prático (README.md) - Segunda parte	Luiz Felipe Gomes Diogo	Luiz Felipe vai complementar escrevendo a segunda parte do README.md focada nos principais comandos e na explicação de como rodar os códigos iniciais/principais do Puppet.
Compiladora da Documentação (PDF)	Mariana Araújo de Souza Monteiro	Responsável pela documentação completa em PDF com a divisão de tarefas e as referências. No documento será reunido o que o grupo pesquisou, listado o que cada um fez e as referências.
Estruturador de Apresentação (Slides - Parte 1)	Lucas Henrique Dias de Medeiros	Focado na apresentação teórica (já que o README é só prático). Vai montar os slides sobre o conceito do Puppet e métodos de uso.
Estruturadora de Apresentação	Nayara Karla Medeiros	Vai complementar os slides falando sobre os principais usos para o mercado. Como a apresentação exige uma metodologia de "sala invertida" (explicar para a turma como se utiliza a ferramenta na prática), ela ajudará a pensar nessa dinâmica visual dos slides.
Revisor de Qualidade (Teste) e Entregas	Vitor Marques de Freitas	Vai testar o "passo a passo" do README criado pelo Otavio e pelo Luiz para ver se faz sentido. Ele também fará o checklist final (Temos o link do repositório? Os slides estão prontos? O PDF está feito? Todo mundo subiu o código?).

2. Método de Uso e Instalação

O Puppet pode ser utilizado em dois modos principais: modo standalone (execução local, sem servidor central, por meio do comando `puppet apply`) e modo agente-servidor (arquitetura cliente-servidor com Puppet Server). A seguir, o passo a passo típico de instalação em distribuições Linux baseadas em Debian/Ubuntu, usando a versão open source (Puppet OSS).

2.1 Pré-requisitos

- Sistema operacional Linux (Ubuntu/Debian, RHEL/CentOS), Windows ou macOS.
- Acesso root/administrador na máquina.
- Conexão à internet para baixar os pacotes.
- Sincronização de data/hora entre os nós (NTP), essencial para validação de certificados SSL.

2.2 Instalação do Puppet Server (nó Master) — Ubuntu/Debian

```
# 1. Baixar e instalar o repositório de pacotes da Puppet
wget https://apt.puppet.com/puppet8-release-jammy.deb
sudo dpkg -i puppet8-release-jammy.deb
sudo apt update
```

```
# 2. Instalar o Puppet Server
sudo apt install -y puppetserver
```

```
# 3. Ajustar memória (opcional, ambiente de testes) e iniciar o serviço
sudo systemctl enable puppetserver
sudo systemctl start puppetserver
sudo systemctl status puppetserver
```

2.3 Instalação do Puppet Agent (nós gerenciados)

```
sudo dpkg -i puppet8-release-jammy.deb
sudo apt update
sudo apt install -y puppet-agent
```

```
# Configurar o endereço do servidor Puppet
sudo /opt/puppetlabs/bin/puppet config set server puppet-master.exemplo.com
```

```
# Habilitar e iniciar o agente
sudo systemctl enable puppet
sudo systemctl start puppet
```

2.4 Assinatura de Certificados (comunicação Agente ↔ Servidor)

A comunicação entre agente e servidor é autenticada via SSL. No primeiro contato, o agente gera uma solicitação de certificado que precisa ser aprovada manualmente no servidor:

```
# No Puppet Server
```

```
sudo /opt/puppetlabs/bin/puppetserver ca list  
sudo /opt/puppetlabs/bin/puppetserver ca sign --certname agente01.exemplo.com
```

2.5 Uso em Modo Standalone (sem servidor)

Para fins de estudo, testes locais ou ambientes pequenos, é possível aplicar manifests diretamente, sem um Puppet Server:

```
# Instalar apenas o pacote puppet (modo standalone)  
sudo apt install -y puppet
```

```
# Criar um manifesto de exemplo (site.pp)  
package { 'nginx':  
  ensure => installed,  
}  
  
service { 'nginx':  
  ensure => running,  
  enable => true,  
  require => Package['nginx'],  
}
```

```
# Validar sintaxe e aplicar  
puppet parser validate site.pp  
sudo puppet apply site.pp
```

2.6 Boas Práticas de Uso

- Versionar todos os manifests e módulos em um repositório Git.
- Separar dados (Hiera) da lógica (manifests) para facilitar reuso entre ambientes (dev, homologação, produção).
- Testar manifests com puppet parser validate e puppet-lint antes de aplicar em produção.
- Utilizar ambientes (environments) para isolar mudanças até que estejam validadas.
- Monitorar execuções e desvios de configuração via PuppetDB e relatórios do Puppet Server.

3. Principais Usos para o Mercado

O Puppet é amplamente adotado por empresas de médio e grande porte, especialmente em setores regulados (financeiro, saúde, governo), devido à sua robustez para gestão de conformidade (compliance) e auditoria em larga escala.

3.1 Aplicações Práticas

- Gestão de configuração em escala: padronizar milhares de servidores (físicos, virtuais ou em nuvem) com o mesmo conjunto de políticas.
- Conformidade e auditoria (compliance): aplicar e comprovar continuamente políticas de segurança exigidas por normas como PCI-DSS, HIPAA e SOX, com relatórios automatizados.
- Correção de desvio de configuração (drift): detectar e reverter automaticamente alterações manuais não autorizadas em servidores.
- Provisionamento consistente de ambientes: garantir que ambientes de desenvolvimento, teste e produção sejam configurados de forma idêntica.
- Integração com pipelines de CI/CD: aplicar configurações como parte de esteiras de entrega contínua, junto a ferramentas como Jenkins e GitLab CI.
- Gestão híbrida e multi-cloud: administrar infraestrutura on-premises e em provedores como AWS, Azure e Google Cloud a partir de um único ponto de controle.

3.2 Setores e Casos de Uso Reais

- Instituições financeiras e bancos: uso para gestão de milhares de servidores com exigências rígidas de compliance (ex.: bancos que gerenciam mais de 7 mil servidores com Puppet).
- Empresas de telecomunicações e provedores de nuvem: padronização de infraestrutura de rede e serviços em larga escala.
- Governo e setor de saúde: ambientes que exigem trilhas de auditoria rígidas e conformidade regulatória contínua.

4. Principais Comandos

A tabela a seguir reúne os comandos mais utilizados no dia a dia com Puppet, cobrindo desde a aplicação de manifests até a gestão de módulos e certificados.

Comando	Descrição
<code>puppet --version</code>	Exibe a versão instalada do Puppet.
<code>puppet resource <tipo></code>	Consulta o estado atual de um tipo de recurso no sistema (ex.: package, service, user).

Comando	Descrição
puppet apply <arquivo.pp>	Aplica um manifesto Puppet localmente, sem precisar do servidor Puppet (Puppet Master).
puppet agent -t	Executa o agente manualmente ("test run"), buscando e aplicando o catálogo do Puppet Server.
puppet agent --enable / --disable	Habilita ou desabilita as execuções automáticas do agente no nó.
puppet config print <parâmetro>	Mostra o valor atual de um parâmetro de configuração (ex.: server, environment).
puppet module install <módulo>	Instala um módulo a partir do Puppet Forge.
puppet module list	Lista os módulos instalados no ambiente atual.
puppet parser validate <arquivo.pp>	Valida a sintaxe de um manifesto sem aplicá-lo.
puppet lookup <chave>	Consulta valores de dados definidos via Hiera.
puppet cert list / puppet cert sign <nó>	Lista certificados pendentes e assina o certificado de um novo agente (gestão de CA).
puppetserver ca list / sign	Equivalente moderno de gestão de certificados no Puppet Server (versões atuais).
puppet describe <tipo>	Mostra a documentação de um tipo de recurso (parâmetros e exemplos).

5. Teste Manifest puppet

5.1 Ambiente de teste

- Máquina virtual criada no **VirtualBox**.
- Instalação do **Ubuntu 26.04 LTS** realizada do zero.
- Ambiente configurado para simular um usuário que nunca utilizou o Puppet.

5.2 Metodologia

- O passo a passo descrito no **README.md** foi seguido integralmente.
- Cada comando foi executado exatamente conforme documentado.
- O objetivo foi verificar se um usuário novo conseguiria reproduzir o processo sem dificuldades.
- Foram realizados testes de **validação de sintaxe**, **simulação** e, quando aplicável, **aplicação real** dos manifests.

5.3 Instalação e configuração do Puppet

- Adição do repositório oficial do Puppet.
- Instalação do pacote **puppet-agent**.
- Configuração da variável de ambiente **PATH**.
- Confirmação da instalação por meio do comando `puppet --version`.
- Instalação concluída com sucesso.

5.4 Resultados dos Manifests Puppet

Vitor — `vitor-exemplo.pp`

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ Aplicação real com `puppet apply`, sem `--noop`: **Aprovado**.
- ★ O manifest instalou e executou o servidor **nginx** com sucesso.
- ★ Funcionamento completo confirmado na prática.

Leonardo — `exemplo_Leo.pp`

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ Nenhum erro identificado.

Nayara — `nayara-exemplo.pp`

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ Nenhum erro identificado.

Otavio — `otavio-exemplo.pp`

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ Nenhum erro identificado.

Mariana — exemplo_MarianaMonteiro.pp

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ O manifest gerencia corretamente:
 - Instalação do nginx;
 - Criação de uma página inicial personalizada;
 - Criação de um usuário no sistema.

Lucas — lucas-exemplo.pp

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Aprovado**.
- ★ O manifest configura corretamente uma tarefa agendada (**cron job**) para realizar backup diário.

Luiz Felipe — luiz-exemplo.pp

- ★ Validação de sintaxe: **Aprovado**.
- ★ Simulação: **Reprovado**.
- ★ O recurso responsável pelo arquivo `/etc/ssh/sshd_config` não pôde ser avaliado.
- ★ O motivo foi a utilização do protocolo `puppet:///`, que depende de uma estrutura de **Puppet Master** centralizado.
- ★ O ambiente de testes utilizou o modo **standalone**, por meio do `puppet apply`, sem um Puppet Master disponível.
- ★ Como consequência, o serviço `sshd` também não pôde ser aplicado devido à dependência entre os recursos.
- ★ O comportamento foi considerado esperado diante da falha anterior na cadeia de recursos.

5.5. Resumo Teste

Integrante	Manifest	Validação de Sintaxe	Simulação	Aplicação/Resultado
Vitor	vitor-exemplo.pp	Aprovado	Aprovado	Aplicado com sucesso; nginx instalado e executado
Leonardo	exemplo_Leo.pp	Aprovado	Aprovado	Funcionou sem erros
Nayara	nayara-exemplo.pp	Aprovado	Aprovado	Funcionou sem erros
Otavio	otavio-exemplo.pp	Aprovado	Aprovado	Funcionou sem erros
Mariana	exemplo_MarianaMonteiro.pp	Aprovado	Aprovado	Nginx, página personalizada e usuário configurados
Lucas	lucas-exemplo.pp	Aprovado	Aprovado	Cron job de backup diário configurado
Luiz Felipe	luiz-exemplo.pp	Aprovado	Falhou	⚠ Uso de <code>puppet:///</code> , dependente de Puppet Master

5.6. Conclusão

Dos **7 manifests** desenvolvidos pelo grupo, **6 funcionaram corretamente do início ao fim**, apresentando resultados satisfatórios tanto na validação de sintaxe quanto na simulação de aplicação.

O manifest de **Luiz Felipe** apresentou uma falha pontual relacionada à origem definida para o arquivo de configuração do SSH. O problema foi identificado e documentado durante a etapa de testes. A falha está relacionada à utilização do protocolo puppet:///, que pressupõe um ambiente com Puppet Master, enquanto os testes foram realizados em modo standalone com puppet apply.

Dessa forma, os testes permitiram validar a reprodução do procedimento descrito no README.md e identificar uma limitação específica de um dos manifests antes da entrega final do trabalho.

6. Referências

PUPPET. About Perforce Puppet. Disponível em: <https://www.puppet.com/about>. Acesso em: 11 ago. 2026.

PERFORCE SOFTWARE. Perforce Puppet: Desired State Automation for Business Velocity. Disponível em: <https://www.perforce.com/products/puppet>. Acesso em: 11 ago. 2026.

PUPPET DOCS. Puppet Documentation. Disponível em: <https://www.puppet.com/docs>. Acesso em: 11 ago. 2026.

PUPPET FORGE. Repositório de módulos Puppet. Disponível em: <https://forge.puppet.com>. Acesso em: 11 ago. 2026.

GARTNER PEER INSIGHTS. Puppet Enterprise Reviews & Ratings. Disponível em: <https://www.gartner.com/reviews/market/devops-platforms/vendor/perforce-software/product/puppet-enterprise>. Acesso em: 11 ago. 2026.